ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM 6 (PAPER 2) · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam 6 (paper 2).

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Programs are not marked strictly on syntax: give marks for correct logic, structure and output. Accept other names, messages and correct ways of solving the problem.

QUESTION 1

1.1 File named correctly with a comment on the first line ✓ (1)

1.2 class Robot: ✓; def __init__(self, name, battery=100): ✓✓; self.name ✓, self.battery ✓, self.distance = 0 ✓ (6)

1.3 def drive(self, km): ✓; needed = km * 2 ✓; if needed > self.battery ✓ print "Battery too low" ✓; else ✓ battery reduced ✓✓ and distance increased ✓ (8)

1.4 def charge(self): ✓ self.battery = 100 ✓ (2)

1.5 def status(self): ✓ return ✓ a string with name, battery and distance ✓✓ (4)

1.6 class DeliveryRobot(Robot): ✓; __init__ that calls super().__init__(…) ✓✓; self.parcels = [] ✓; def load(self, parcel): ✓ append ✓ (6)

1.7 Object created ✓; two parcels loaded and 10 km driven ✓; status and parcels displayed ✓ (3)

```
# YourName - Question 1
class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery
        self.distance = 0

    def drive(self, km):
        needed = km * 2                         # 2 % per km
        if needed > self.battery:
            print("Battery too low")
        else:
            self.battery -= needed
            self.distance += km

    def charge(self):
        self.battery = 100

    def status(self):
        return f"{self.name}: battery {self.battery}%, {self.distance} km driven"

class DeliveryRobot(Robot):
    def __init__(self, name, battery=100):
        super().__init__(name, battery)
        self.parcels = []

    def load(self, parcel):
        self.parcels.append(parcel)

d1 = DeliveryRobot("D1")
d1.load("Parcel A")
d1.load("Parcel B")
d1.drive(10)
```

```
print(d1.status())          # D1: battery 80%, 10 km driven
print(d1.parcels)
```

[30]

QUESTION 2

2.1 File named correctly ✓ (1)

2.2 def save_reading(time, temp): ✓; open in append mode "a" ✓✓; write time, comma, temp ✓ and a newline ✓ (5)

2.3 Loop that runs 3 times ✓; input of time ✓ and temperature ✓; save_reading called ✓✓ (5)

2.4 def load_readings(): ✓; try ✓ open and read the lines ✓; split on the comma ✓; temperature converted to float ✓; tuples added to a list and returned ✓; except FileNotFoundError ✓ with the message and an empty list ✓ (8)

2.5 Every reading displayed ✓✓; average = total ÷ number ✓✓ (guarded against an empty list); highest temperature found ✓✓ with its time ✓; clear output ✓ (8)

2.6 Program ✓, printout with line numbers ✓ and output screenshot ✓ (3)

```
# YourName - Question 2
def save_reading(time, temp):
    with open("temps.txt", "a") as f:
        f.write(f"{time},{temp}\n")

def load_readings():
    readings = []
    try:
        with open("temps.txt") as f:
            for line in f:
                time, temp = line.strip().split(",")
                readings.append((time, float(temp)))
    except FileNotFoundError:
        print("No readings yet")
        return []
    return readings

for i in range(3):
    t = input("Time (hh:mm): ")
    temp = float(input("Temperature (°C): "))
    save_reading(t, temp)

data = load_readings()
if data:
    for t, temp in data:
        print(t, temp)
    average = sum(temp for t, temp in data) / len(data)
    hottest = max(data, key=lambda r: r[1])
    print(f"Average: {average:.2f} °C")
    print(f"Highest: {hottest[1]} °C at {hottest[0]}")
```

[30]

QUESTION 3

3.1 File named correctly ✓ (1)

3.2 Dictionary named members ✓ with the three correct members and levels ✓✓ (3)

3.3 def add_member(): ✓; name input ✓; loop that repeats until valid ✓; try ✓ int(input()) ✓ except ValueError ✓; range check 1–4 ✓; member added with a message ✓ (8)

3.4 def remove_member(): ✓; name input ✓; if name in members ✓ del / pop ✓; else message ✓ (5)

3.5 def search_member(): ✓; if found, display the level ✓✓; else message ✓ (4)

3.6 def list_members(): ✓; for loop ✓ over sorted(members) ✓✓; name and level displayed ✓ (5)

3.7 while loop ✓; menu displayed ✓ and choice input ✓; choices 1–4 call the correct functions ✓✓✓✓; choice 5 displays "Goodbye!" and breaks ✓ (8)

3.8 else: print("Invalid choice.") ✓✓ (2)

3.9 Output matches the example ✓✓ (2)

3.10 Saved ✓ and printed with line numbers ✓ (2)

```
# YourName - Question 3
members = {"Thabo": 1, "Bongani": 2, "Naledi": 4}

def add_member():
    name = input("Member name: ").strip()
    while True:
        try:
            level = int(input("Level (1-4): "))
            if 1 <= level <= 4:
                break
        except ValueError:
            pass
        print("Please enter a number from 1 to 4.")
    members[name] = level
    print(f"{name} added at level {level}.")

def remove_member():
    name = input("Name to remove: ").strip()
    if name in members:
        del members[name]
        print(f"{name} removed.")
    else:
        print(f"{name} is not a member.")

def search_member():
    name = input("Name to search: ").strip()
    if name in members:
        print(f"{name} is at level {members[name]}.")
    else:
        print(f"{name} is not a member.")

def list_members():
    for name in sorted(members):
        print(f"{name} - level {members[name]}")

while True:
```

```
    print("1 Add  2 Remove  3 Search  4 List  5 Quit")
    choice = input("Choice: ")
    if choice == "1":
        add_member()
    elif choice == "2":
        remove_member()
    elif choice == "3":
        search_member()
    elif choice == "4":
        list_members()
    elif choice == "5":
        print("Goodbye!")
        break
    else:
        print("Invalid choice.")
```

[40]

TOTAL: 100

END OF MARKING GUIDELINE